



Laboratorio No.2 Semana #13 – API Secrets y autenticación

Nombre: Bernardette Sibila Aragón Leóm Carné: 1265425

Objetivo: Practicar el consumo de una API REST autenticada y aplicar una buena práctica básica de seguridad: evitar colocar credenciales directamente en el código fuente. El estudiante utilizará Postman, una aplicación C# y GitHub.

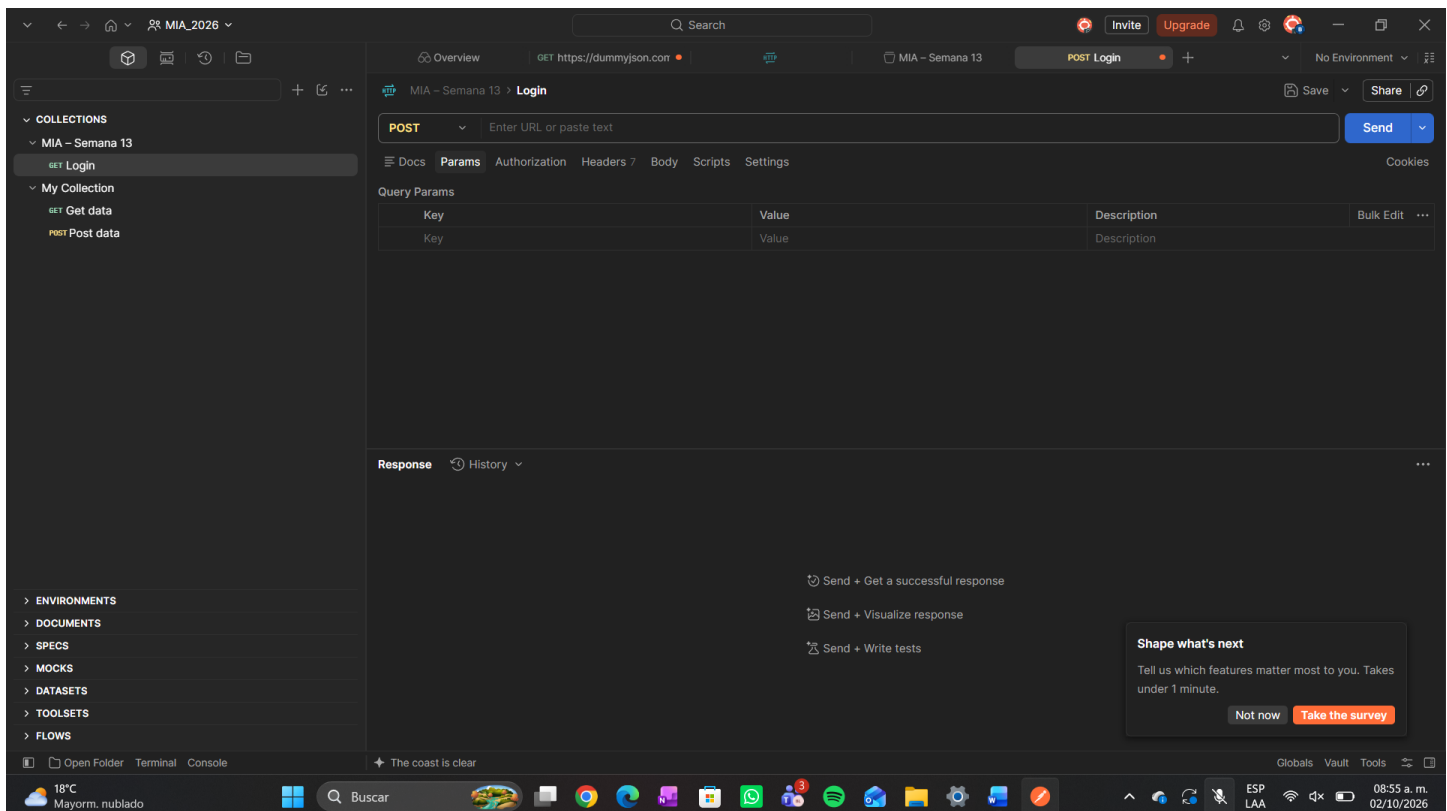
Parte 1 – Probar autenticación con Postman

Para esta práctica se utilizará DummyJSON, una API pública de pruebas que permite realizar un inicio de sesión y obtener un token de acceso.

Documentación: <https://dummyjson.com/docs/auth> <https://dummyjson.com/docs/users>

1. Crear la solicitud de inicio de sesión

- Abra Postman y cree una nueva colección llamada: MIA – Semana 13.
- Cree una solicitud llamada: Login.
- Seleccione el método POST.
- Utilice el endpoint: `https://dummyjson.com/auth/login`
- En Body seleccione raw y JSON.
Ingrese las credenciales de prueba: Username: emilys | Password: emilyspass.
`{ "username": "emilys", "password": "emilyspass" }`
- Ejecute la solicitud y localice en la respuesta el campo accessToken.





The screenshot shows the Postman interface with a POST request to `https://dummyjson.com/auth/login`. The request body is a JSON object: `{ "username": "emilys", "password": "emilyspass" }`. The response is a 200 OK status with a response time of 226 ms and a body size of 2.3 KB. The response body is a JSON object:

```
{  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXN1cm5hbWU1OiJlbWlseXN1LCJlbWpbcCI6ImVtaWw5LmpvaG5zb25AeC5kdW1teWpzb24uY29tIiwiaWZmIjoiRmVpbHkiLCJzYXN0TmFtZSI6IkpvaG5zb24iLCJnZW5kZXI6ImZw1hbGU1LCJpbWFnZSI6Imh0dHBz0i8vZHVtbXlqc29uLmNvbS9pY29uL2VtaWw5cy8xMjgiLCJpYXQ1OiJlE30TA5NTMwNTgsImV4cCI6MTc5MDk1NjY1OH0.4Ync338kuoIj0brihUZ87A2jB1u0Tb6pgQH20IF7kgVI",  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXN1cm5hbWU1OiJlbWlseXN1LCJlbWpbcCI6ImVtaWw5LmpvaG5zb25AeC5kdW1teWpzb24uY29tIiwiaWZmIjoiRmVpbHkiLCJzYXN0TmFtZSI6IkpvaG5zb24iLCJnZW5kZXI6ImZw1hbGU1LCJpbWFnZSI6Imh0dHBz0i8vZHVtbXlqc29uLmNvbS9pY29uL2VtaWw5cy8xMjgiLCJpYXQ1OiJlE30TA5NTMwNTgsImV4cCI6MTc5MDk1NjY1OH0.q78S0-HL5Ism38-uoQ-2Lh12dfQ2GSPUACTz6rBsc6s",  "id": 1,  "username": "emilys",  "email": "emily.johnson@x.dummyjson.com",  "firstName": "Emily",  "lastName": "Johnson"}
```

2. Utilizar token

- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: `https://dummyjson.com/user/me`
- En Authorization seleccione Bearer Token.
- Copie el accessToken obtenido en la solicitud Login.
- Ejecute la solicitud.
- Verifique que la API devuelve la información del usuario autenticado.

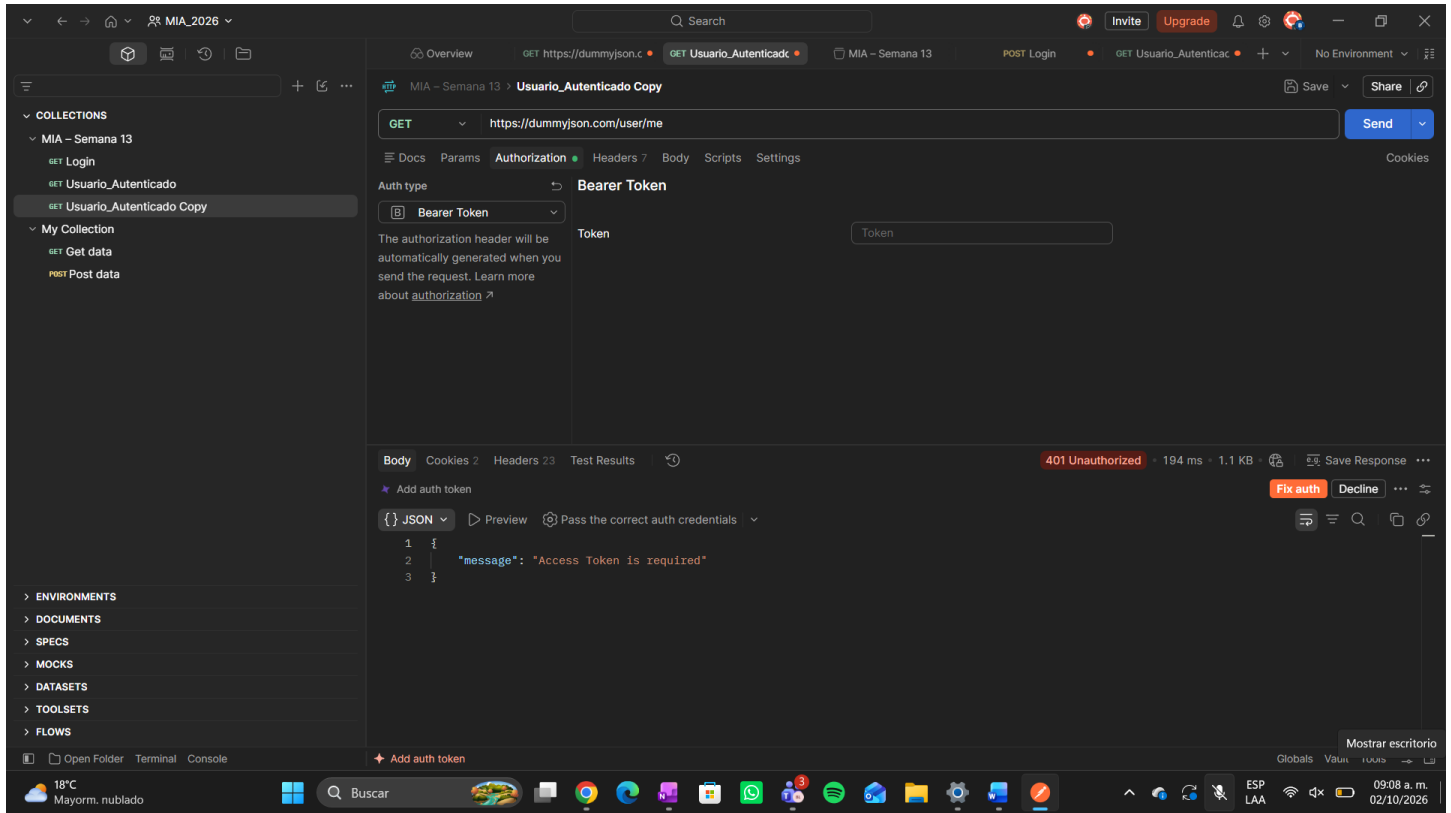


The screenshot shows the Postman application interface. The top bar includes navigation icons, a search bar, and buttons for 'Invite' and 'Upgrade'. The left sidebar displays a collection named 'MIA - Semana 13' with a sub-collection 'Usuario_Autenticado' containing a 'Login' endpoint. The main workspace shows a GET request to 'https://dummyjson.com/user/me' with a 'Bearer Token' authorization type. The response is a 200 OK status with a JSON body containing user details for Emily Johnson.

```
{
  "id": 1,
  "firstName": "Emily",
  "lastName": "Johnson",
  "maidenName": "Smith",
  "age": 29,
  "gender": "female",
  "email": "emily.johnson@x.dummyjson.com",
  "phone": "+81 965-431-3024",
  "username": "emilys",
  "password": "emilyspass",
  "birthDate": "1996-5-30",
  "image": "https://dummyjson.com/icon/emilys/128",
  "bloodGroup": "O-"
}
```

3. Comprobar qué ocurre sin autenticación

- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.
- Compare la respuesta con la solicitud autenticada.
- Registre el código HTTP y explique la diferencia.



El Bearer Token sirve para demostrar que el usuario ya inició sesión y tiene permiso para acceder a determinados recursos de la API. Este token se obtiene al realizar el login y luego se envía en el encabezado Authorization de las solicitudes. Si la solicitud incluye un token válido, la API permite el acceso; si se realiza sin token, puede rechazar la solicitud.

4. Evidencia

- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.
- Captura de la prueba sin token.
- Indique qué función cumple el Bearer Token.

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.



Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- Abra el proyecto C# de Azure Storage.
- En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- Busque la variable que contiene la Connection String.
- Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada:

AZURE_STORAGE_CONNECTION_STRING

La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

```
1 using Azure.Storage.Blobs;
2 using Azure.Storage.Blobs.Models;
3
4 string? connectionString =
5     Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING");
6
7 if (string.IsNullOrWhiteSpace(connectionString))
8 {
9     Console.WriteLine("No se encontró la Connection String.");
10    Console.WriteLine("Configure la variable de entorno AZURE_STORAGE_CONNECTION_STRING.");
11    return;
12 }
13
14 string containerName = "miaarchivos";
15
16 BlobServiceClient blobServiceClient =
17     new BlobServiceClient(connectionString);
18
19 BlobContainerClient containerClient =
20     blobServiceClient.GetBlobContainerClient(containerName);
21
22 await containerClient.CreateIfNotExistsAsync();
```

Terminal output:

```
PS C:\mia_semana12\MIA_AzureBlob_BN>
```

7. Crear la variable para ejecutar localmente

- En la carpeta principal del proyecto, cree un archivo llamado .env.
- Dentro del archivo agregue la variable:
AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING
- Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- Compruebe que el programa continúa funcionando correctamente.
- No publique el archivo .env en GitHub**, ya que contiene la credencial real.



The screenshot shows the Visual Studio Code interface with the `.env` file open in the Explorer. The file contains the following content:

```
1 AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING
2
```

The Terminal panel shows the following commands and output:

```
PS C:\Users\berna\OneDrive\Escritorio\url-mia-2026\semana-12\lab12\mia_semana12\MIA_AzureBlob_BN> git status
PS C:\Users\berna\OneDrive\Escritorio\url-mia-2026\semana-12\lab12\mia_semana12\MIA_AzureBlob_BN> git commit -m "Laboratorio 2 Semana 13"
[master dbb06a4] Laboratorio 2 Semana 13
4 files changed, 8 insertions(+), 2 deletions(-)
create mode 100644 .gitignore
create mode 100644 semana-11/.gitkeep
create mode 100644 semana-9/.gitkeep
PS C:\Users\berna\OneDrive\Escritorio\url-mia-2026\semana-12\lab12\mia_semana12\MIA_AzureBlob_BN> git push origin master
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
```

The status bar at the bottom indicates the file is in the `MIA_AzureBlob_BN` project, using the `C#` workspace, and the file encoding is `UTF-8`.

The screenshot shows the Visual Studio Code interface with the `.env.production` file open in the Explorer. The file contains the following content:

```
1 AZURE_STORAGE_CONNECTION_STRING_LOCAL=${AZURE_STORAGE_CONNECTION_STRING_GITHUB}
```

The Terminal panel shows the following commands and output:

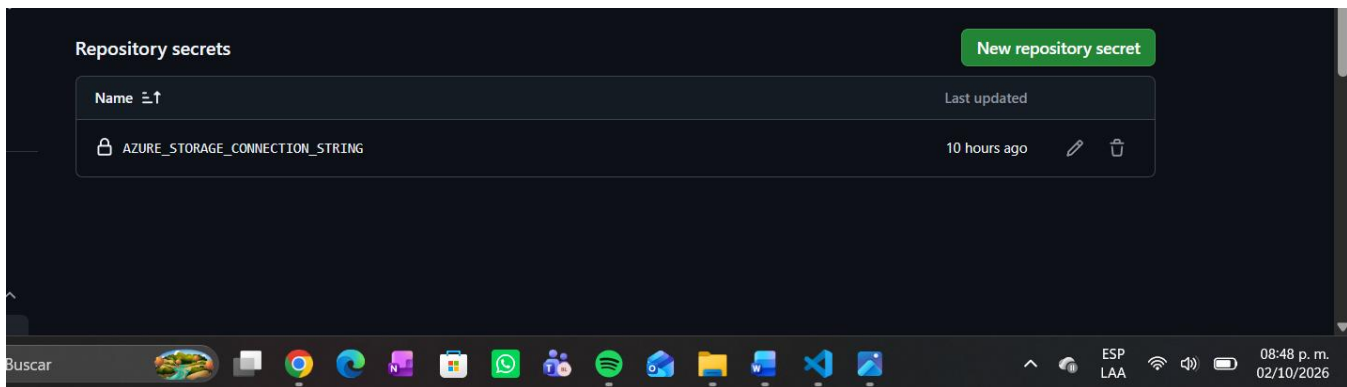
```
PS C:\Users\berna\OneDrive\Escritorio\url-mia-2026\semana-12\lab12\mia_semana12\MIA_AzureBlob_BN> git status
PS C:\Users\berna\OneDrive\Escritorio\url-mia-2026\semana-12\lab12\mia_semana12\MIA_AzureBlob_BN> git commit -m "Laboratorio 2 Semana 13"
[master dbb06a4] Laboratorio 2 Semana 13
4 files changed, 8 insertions(+), 2 deletions(-)
create mode 100644 .gitignore
create mode 100644 semana-11/.gitkeep
create mode 100644 semana-9/.gitkeep
PS C:\Users\berna\OneDrive\Escritorio\url-mia-2026\semana-12\lab12\mia_semana12\MIA_AzureBlob_BN> git push origin master
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
```

The status bar at the bottom indicates the file is in the `MIA_AzureBlob_BN` project, using the `C#` workspace, and the file encoding is `UTF-8`.

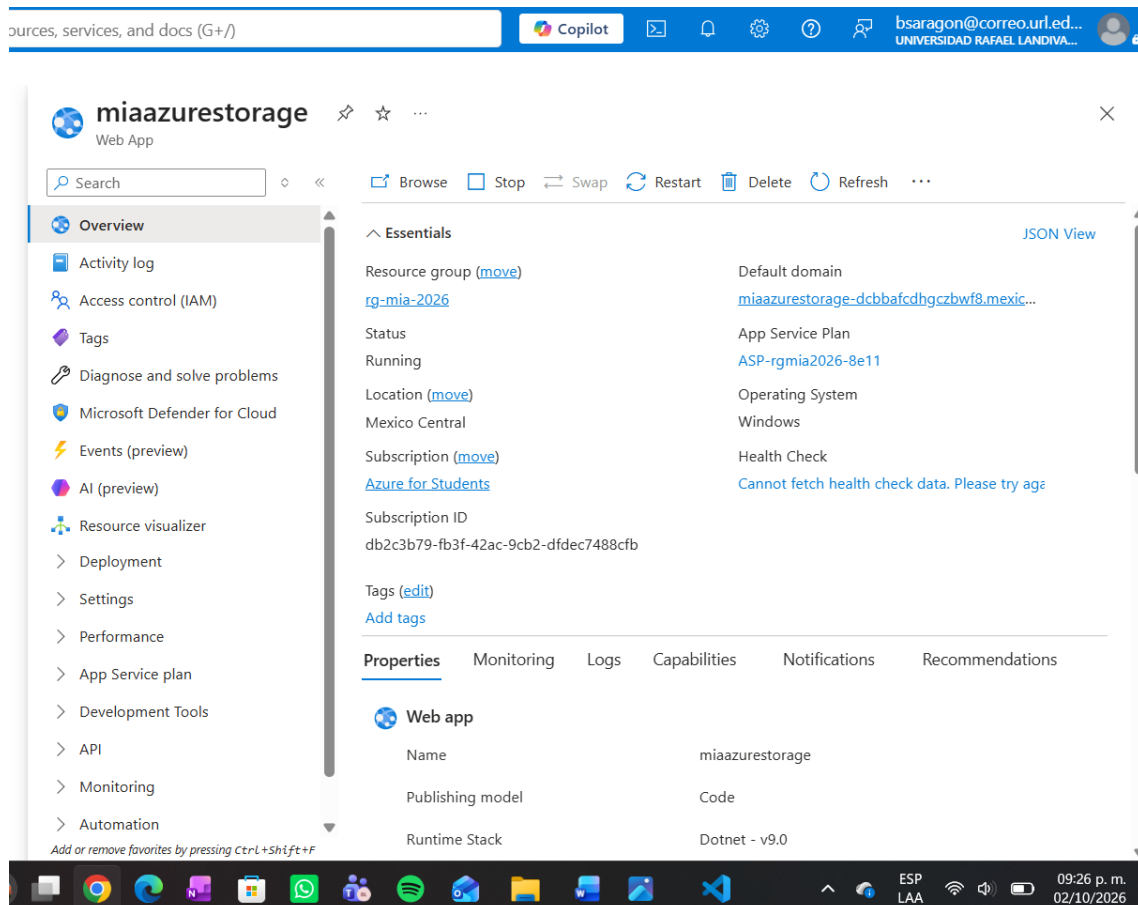


8. Crear el Secret en Github

- Ingrese al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: AZURE_STORAGE_CONNECTION_STRING
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.



Azure





Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.